

Practice Set 1 – Basics + CRUD + Filtering

Scenario: Restaurant Menu

```
CREATE DATABASE sql_practice_pack;
USE sql_practice_pack;

CREATE TABLE menu_items (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(100),
    category VARCHAR(50),
    price DECIMAL(10,2),
    available_qty INT
);

INSERT INTO menu_items VALUES
(1, 'Chicken Biryani', 'Main Course', 320, 25),
(2, 'Paneer Tikka', 'Starter', 240, 15),
(3, 'Masala Dosa', 'Breakfast', 120, 30),
(4, 'Veg Burger', 'Fast Food', 180, 10),
(5, 'Cold Coffee', 'Beverage', 150, 20),
(6, 'Chicken Burger', 'Fast Food', 220, 8),
(7, 'Idli', 'Breakfast', 80, 40),
(8, 'Fresh Lime', 'Beverage', 90, 0);
```

Exercises

1. Display all menu items.
2. Display only item name and price.
3. Add a new menu item.
4. Change the price of `Chicken Biryani`.
5. Increase the price of all `Fast Food` products by 10%.
6. Reduce the available quantity of `Veg Burger` by 2.
7. Delete products whose available quantity is 0.
8. Display items costing more than ₹200.
9. Display items priced between ₹100 and ₹250.
10. Display all Breakfast items.

11. Display items belonging to Breakfast or Beverage.
 12. Display products whose names contain `Chicken`.
 13. Sort items from highest price to lowest.
 14. Display the three most expensive items.
 15. Display products where available quantity is less than 15.
-

Practice Set 2 – Aggregate Functions + GROUP BY + HAVING

Scenario: Food Delivery Orders

```
CREATE TABLE food_orders (  
    order_id INT PRIMARY KEY,  
    restaurant VARCHAR(100),  
    city VARCHAR(50),  
    food_type VARCHAR(50),  
    order_amount DECIMAL(10,2),  
    delivery_partner VARCHAR(50),  
    order_date DATE  
);
```

```
INSERT INTO food_orders VALUES  
(101, 'Spice Hub', 'Hyderabad', 'Indian', 850, 'Ravi', '2026-09-01'),  
(102, 'Burger Zone', 'Hyderabad', 'Fast Food', 520, 'Kiran', '2026-09-01'),  
(103, 'Pizza Point', 'Mumbai', 'Fast Food', 1100, 'Ravi', '2026-09-02'),  
(104, 'Curry House', 'Bangalore', 'Indian', 760, 'Aman', '2026-09-02'),  
(105, 'Spice Hub', 'Hyderabad', 'Indian', 1250, 'Kiran', '2026-09-03'),  
(106, 'Sushi World', 'Mumbai', 'Japanese', 1800, 'Aman', '2026-09-03'),  
(107, 'Pizza Point', 'Mumbai', 'Fast Food', 900, 'Ravi', '2026-09-04'),  
(108, 'Curry House', 'Bangalore', 'Indian', 640, 'Kiran', '2026-09-04'),  
(109, 'Burger Zone', 'Hyderabad', 'Fast Food', 430, 'Aman', '2026-09-05'),  
(110, 'Sushi World', 'Mumbai', 'Japanese', 2100, 'Ravi', '2026-09-05'),  
(111, 'Spice Hub', 'Hyderabad', 'Indian', 950, 'Aman', '2026-09-06'),  
(112, 'Curry House', 'Bangalore', 'Indian', 880, 'Ravi', '2026-09-06');
```

Exercises

1. Find the total number of orders.

2. Find total revenue.
 3. Find average order value.
 4. Find highest order amount.
 5. Find lowest order amount.
 6. Find total revenue by city.
 7. Find number of orders by restaurant.
 8. Find average order value by food type.
 9. Find total revenue generated by each delivery partner.
 10. Find cities having more than 3 orders.
 11. Find restaurants generating more than ₹2,000 revenue.
 12. Find delivery partners whose average order amount exceeds ₹800.
 13. Find food types whose total revenue exceeds ₹2,500.
 14. Display city-wise revenue from highest to lowest.
 15. Find the top-performing restaurant based on revenue.
-

Practice Set 3 – JOIN Practice

Scenario: Training Institute

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(100),  
    city VARCHAR(50)  
);
```

```
INSERT INTO students VALUES  
(1, 'Arun', 'Hyderabad'),  
(2, 'Megha', 'Mumbai'),  
(3, 'Zaid', 'Hyderabad'),  
(4, 'Pooja', 'Pune'),  
(5, 'Rohan', 'Delhi'),  
(6, 'Sana', NULL),  
(7, 'Vijay', 'Bangalore');
```

```
CREATE TABLE courses (  
    course_id INT PRIMARY KEY,  
    course_name VARCHAR(100),  
    fee DECIMAL(10,2)  
);
```

```
INSERT INTO courses VALUES
(101, 'Python', 15000),
(102, 'Data Engineering', 25000),
(103, 'Power BI', 12000),
(104, 'Cloud Computing', 20000),
(105, 'Cyber Security', 22000),
(106, 'Machine Learning', 28000);
```

```
CREATE TABLE enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    enrollment_date DATE
);
```

```
INSERT INTO enrollments VALUES
(1001, 1, 101, '2026-09-01'),
(1002, 1, 102, '2026-09-03'),
(1003, 2, 103, '2026-09-04'),
(1004, 3, 102, '2026-09-05'),
(1005, 4, 104, '2026-09-06'),
(1006, 2, 101, '2026-09-07'),
(1007, 3, 105, '2026-09-08'),
(1008, 20, 102, '2026-09-09'),
(1009, 5, NULL, '2026-09-10');
```

Exercises

1. Display student name and course name.
2. Display student name, city, course name and fee.
3. Find students enrolled in **Data Engineering**.
4. Display all students including those having no enrollment.
5. Find students who have never enrolled in a course.
6. Display all courses including courses having no students.
7. Find courses having no enrollments.
8. Identify enrollment records having no valid student.
9. Identify enrollment records having no valid course.
10. Find total courses taken by each student.
11. Find total fees associated with each student's enrollments.
12. Find students enrolled in more than one course.
13. Find courses having more than one student.

14. Find total enrollment revenue by course.
 15. Display the course generating the highest enrollment revenue.
-

Practice Set 4 – Stored Procedures

Scenario: Vehicle Rental

```
CREATE TABLE vehicles (  
    vehicle_id INT PRIMARY KEY,  
    vehicle_name VARCHAR(100),  
    vehicle_type VARCHAR(50),  
    daily_rate DECIMAL(10,2),  
    available_status VARCHAR(20)  
);  
  
INSERT INTO vehicles VALUES  
(1, 'Honda City', 'Car', 2500, 'Available'),  
(2, 'Toyota Innova', 'Car', 3500, 'Available'),  
(3, 'Royal Enfield', 'Bike', 1200, 'Rented'),  
(4, 'Activa', 'Scooter', 700, 'Available'),  
(5, 'Mahindra Thar', 'SUV', 4500, 'Rented'),  
(6, 'Hyundai Creta', 'SUV', 3200, 'Available'),  
(7, 'KTM Duke', 'Bike', 1500, 'Available');
```

Stored Procedure Exercises

1. Create `GetAllVehicles`.
 2. Create `GetAvailableVehicles`.
 3. Create a procedure accepting vehicle type and returning matching vehicles.
 4. Create a procedure accepting maximum daily rate.
 5. Create a procedure accepting vehicle ID and new daily rate.
 6. Create a procedure to change vehicle status.
 7. Create a procedure that increases the daily rate by a supplied percentage.
 8. Create a procedure that deletes a vehicle based on vehicle ID.
 9. Create a procedure returning all vehicles between two rental rates.
 10. Create a procedure that counts vehicles based on vehicle type.
-

Practice Set 5 – Data Cleansing + RegEx

Scenario: Online Registration Data

```
CREATE TABLE registrations (  
    registration_id INT PRIMARY KEY,  
    full_name VARCHAR(100),  
    email VARCHAR(100),  
    mobile VARCHAR(40),  
    city VARCHAR(50),  
    postal_code VARCHAR(20)  
);  
  
INSERT INTO registrations VALUES  
(1, ' rohit sharma ', ' ROHIT@GMAIL.COM ', '+91-98765-43210', 'hyderabad',  
(2, 'SARA KHAN', 'sara@yahoo.com', '99887 66554', ' MUMBAI ', '400001'),  
(3, ' amit patel ', '', '(040)99887766', 'Hyderabad', '500 032'),  
(4, 'Neha Singh ', NULL, '9876543210', 'BANGALORE', '560001'),  
(5, 'imran ali', 'IMRAN@MAIL.COM ', '91 9988772211', NULL, '500084'),  
(6, 'Priya Rao', 'priya@gmail', '98765-AB210', 'Pune', '411001');
```

Exercises

1. Remove spaces before and after names.
 2. Convert names to uppercase.
 3. Convert email addresses to lowercase.
 4. Convert blank email values into `NULL`.
 5. Remove spaces and hyphens from mobile numbers.
 6. Remove every non-numeric character from mobile numbers.
 7. Standardize city names to uppercase.
 8. Find records where city is `NULL`.
 9. Find records where email is `NULL` or blank.
 10. Find only Gmail email addresses using RegEx.
 11. Identify incorrectly formatted email addresses.
 12. Extract only numeric characters from postal codes.
 13. Find mobile numbers containing alphabetic characters.
 14. Produce a cleaned output containing name, email, mobile and city.
 15. Create a cleaned copy of the data in another table.
-

Practice Set 6 – Analytical Functions + OVER + PARTITION BY

Scenario: Call Centre Performance

```
CREATE TABLE call_performance (  
    call_id INT PRIMARY KEY,  
    agent_name VARCHAR(100),  
    team VARCHAR(50),  
    calls_handled INT,  
    customer_rating DECIMAL(3,2),  
    performance_date DATE  
);  
  
INSERT INTO call_performance VALUES  
(1, 'Aman', 'Alpha', 42, 4.50, '2026-09-01'),  
(2, 'Sara', 'Alpha', 38, 4.70, '2026-09-01'),  
(3, 'Ravi', 'Beta', 50, 4.20, '2026-09-01'),  
(4, 'Neha', 'Beta', 45, 4.80, '2026-09-01'),  
(5, 'Aman', 'Alpha', 48, 4.60, '2026-09-02'),  
(6, 'Sara', 'Alpha', 44, 4.50, '2026-09-02'),  
(7, 'Ravi', 'Beta', 46, 4.30, '2026-09-02'),  
(8, 'Neha', 'Beta', 52, 4.90, '2026-09-02'),  
(9, 'Kabir', 'Alpha', 41, 4.40, '2026-09-01'),  
(10, 'Kabir', 'Alpha', 49, 4.60, '2026-09-02'),  
(11, 'Pooja', 'Beta', 45, 4.70, '2026-09-01'),  
(12, 'Pooja', 'Beta', 50, 4.80, '2026-09-02');
```

Exercises

1. Display each row along with total calls handled across the entire table.
2. Display each row along with total calls handled by its team.
3. Display each agent's record along with average calls handled by the team.
4. Display each row along with team average customer rating.
5. Find cumulative calls handled by each agent ordered by date.
6. Calculate cumulative calls at the team level.
7. Compare each row's calls with the team's average calls.
8. Display the previous day's call count for each agent.

9. Calculate the difference between current and previous call count.
 10. Display total calls handled by each agent without collapsing rows.
-

Practice Set 7 – Ranking

Use the same `call_performance` table.

Exercises

1. Rank all records based on calls handled.
 2. Use `ROW_NUMBER()` to number records from highest to lowest calls.
 3. Use `RANK()` to rank records.
 4. Use `DENSE_RANK()` to rank records.
 5. Compare the results of all three ranking functions.
 6. Rank agents separately within each team.
 7. Rank records according to customer rating.
 8. Find the top 3 performance records in each team.
 9. Find each agent's best performance day.
 10. Rank agents based on their total calls handled.
-

Practice Set 8 – CTE + Correlated Subqueries

Scenario: Insurance Claims

```
CREATE TABLE insurance_claims (  
    claim_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    insurance_type VARCHAR(50),  
    claim_amount DECIMAL(12,2),  
    branch VARCHAR(50)  
);
```

```
INSERT INTO insurance_claims VALUES  
(1, 'Ajay Kumar', 'Health', 75000, 'Hyderabad'),  
(2, 'Meena Shah', 'Motor', 45000, 'Mumbai'),  
(3, 'Rohit Jain', 'Health', 125000, 'Hyderabad'),
```

```
(4, 'Sara Ali', 'Travel', 30000, 'Delhi'),
(5, 'Vikas Rao', 'Motor', 85000, 'Mumbai'),
(6, 'Nisha Singh', 'Health', 60000, 'Delhi'),
(7, 'Imran Khan', 'Travel', 55000, 'Hyderabad'),
(8, 'Pooja Patel', 'Motor', 40000, 'Delhi'),
(9, 'Karan Mehta', 'Health', 150000, 'Mumbai'),
(10, 'Farah Ahmed', 'Travel', 35000, 'Hyderabad');
```

CTE Exercises

1. Create a CTE calculating total claims by insurance type.
2. Use a CTE to calculate total claims by branch.
3. Find insurance types whose total claim amount exceeds ₹2,00,000.
4. Create a CTE calculating average claim amount and filter claims above that average.
5. Create a CTE and rank insurance types by total claim amount.
6. Create two CTEs in the same query and combine their results.

Correlated Subquery Exercises

7. Find claims greater than the average claim amount for their own insurance type.
 8. Find claims greater than the average claim amount at their own branch.
 9. Find the largest claim for each insurance type.
 10. Find customers whose claim is above the branch average.
-

Practice Set 9 – Hierarchical Queries

Scenario: Company Reporting Structure

```
CREATE TABLE staff_hierarchy (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(100),
    manager_id INT,
    designation VARCHAR(100)
);

INSERT INTO staff_hierarchy VALUES
(1, 'Raj Malhotra', NULL, 'CEO'),
(2, 'Meera Shah', 1, 'CTO'),
(3, 'Vikram Rao', 1, 'Sales Director'),
```

```
(4, 'Aman Khan', 2, 'Engineering Manager'),  
(5, 'Sara Ali', 2, 'Data Manager'),  
(6, 'Rohit Das', 4, 'Developer'),  
(7, 'Priya Singh', 4, 'Developer'),  
(8, 'Kabir Ahmed', 5, 'Data Engineer'),  
(9, 'Neha Rao', 5, 'Data Analyst'),  
(10, 'Imran Sheikh', 3, 'Sales Manager'),  
(11, 'Pooja Jain', 10, 'Sales Executive');
```

Exercises

1. Find employees having no manager.
 2. Find direct reports of the CEO.
 3. Find direct reports of the CTO.
 4. Use a recursive CTE to display the complete hierarchy.
 5. Add a hierarchy level to the result.
 6. Find all employees working below **Meera Shah**.
 7. Find everyone working below **Aman Khan**.
 8. Display employee → manager relationships.
 9. Determine how many employees are at each hierarchy level.
 10. Display the organization starting with CEO and recursively moving downward.
-

Practice Set 10 – GROUP BY Extensions / ROLLUP

Scenario: Hotel Revenue

```
CREATE TABLE hotel_bookings (  
    booking_id INT PRIMARY KEY,  
    hotel_city VARCHAR(50),  
    room_type VARCHAR(50),  
    nights INT,  
    amount DECIMAL(10,2)  
);  
  
INSERT INTO hotel_bookings VALUES  
(1, 'Hyderabad', 'Standard', 2, 6000),
```

```
(2, 'Hyderabad', 'Deluxe', 3, 13500),  
(3, 'Hyderabad', 'Suite', 2, 18000),  
(4, 'Mumbai', 'Standard', 2, 9000),  
(5, 'Mumbai', 'Deluxe', 3, 18000),  
(6, 'Mumbai', 'Suite', 1, 15000),  
(7, 'Bangalore', 'Standard', 3, 10500),  
(8, 'Bangalore', 'Deluxe', 2, 12000),  
(9, 'Bangalore', 'Suite', 2, 20000);
```

Exercises

1. Find total revenue by city.
 2. Find total revenue by room type.
 3. Group by city and room type.
 4. Produce city-wise subtotals using `WITH ROLLUP`.
 5. Produce a grand total.
 6. Calculate total nights by city and room type.
 7. Display subtotals and overall totals in the same result.
-